

Part 2

Problem 1

(1) $\lceil 1024/100 \rceil = \lceil 10.24 \rceil = 10$ records per block

(2) $\lceil 100000/10 \rceil = 10000$ blocks

Blocks per track: Each track has 50 sectors $\times$ 512 bytes/sector = 25,600 bytes per track.

$\lceil (25,600 \text{ bytes/track}) / (1024 \text{ bytes/block}) \rceil = 25$ blocks/track

Tracks needed: $\lceil 10000 \text{ blocks} / (25 \text{ blocks/track}) \rceil = 400$ tracks

Surface needed: Each surface has 2,000 tracks, so: $\lceil 400 \text{ tracks} / (2000 \text{ tracks/surface}) \rceil = 1$ surface

(3) $10 \text{ surfaces} \times 2000 \text{ tracks} \times 50 \text{ sectors} \times 512 \text{ bytes} = 512000000$ bytes

Total blocks on disk: $512000000 / 1024 = 500000$ blocks

Record per block: $500000 \times 10 = 5000000$ records

(4) Seek time: 10 msec

Average rotational latency : $1/2 \times 1 \text{ revolution} / 5400 \text{ rpm} \approx 11.11 \times 0.5 = 5.56 \text{ msec}$

Total transfer time $(10000 \text{ blocks} \times 1024 \text{ bytes}) / (2304000 \text{ bytes/sec}) \approx 4.44 \text{ s}$

Total time = 10 msec + 5.56 msec + 4.44s = 4.46s

Problem 2

Way 1: Mark Record 5 as deleted (leave an empty slot)

Idea: Since this is a heap file with fixed-length records, one simple way is to delete the record logically by marking its slot as empty (or deleted), without moving any other records. This keeps all other record positions unchanged.

Slot	SID	Sname	Major	Status
Record 0	G-743	Smith	CS	occupied
Record 1	G-142	Park	Math	occupied
Record 2	G-921	Brown	Math	occupied
Record 3	G-541	Li	IS	occupied
Record 4	G-032	Zhang	CS	occupied
Record 5	—	—	—	empty / deleted
Record 6	G-715	Miller	IS	occupied
Record 7	G-173	Kim	EE	occupied
Record 8	G-458	Kapur	CS	occupied

Explanation: In this method, the system simply marks Record 5 as deleted. The space of Record 5 becomes a free slot that can later be reused for insertion of a new record.

Advantage:

- ① Very simple deletion
- ② No need to move other records
- ③ Record IDs / slot numbers of other records remain unchanged

Disadvantage:

- ① Leaves a hole in the file
- ② Over time, many deletions may cause wasted space

Way 2: Replace Record 5 with the last record, then remove the last slot

Idea: Keep the file compact by moving the last record into the deleted position. Here, the last record is Record 8: (G-458,Kapur,CS). So we copy Record 8 into Record 5, then remove Record 8.

Slot	SID	Sname	Major	Status
Record 0	G-743	Smith	CS	occupied
Record 1	G-142	Park	Math	occupied
Record 2	G-921	Brown	Math	occupied
Record 3	G-541	Li	IS	occupied
Record 4	G-032	Zhang	CS	occupied
Record 5	G-458	Kapur	CS	occupied
Record 6	G-715	Miller	IS	occupied
Record 7	G-173	Kim	EE	occupied

Explanation: In this method, the deleted slot is filled by the last record in the file. Then the last slot is removed, so the file stays compact and no empty hole remains.

Advantage:

①No wasted space inside the file

②File remains compact

Disadvantage:

①Changes the position of the moved record

②If some index or pointer refers to Record 8, it must be updated because Kapur is now in Record

③ If any indices or pointers exist that point to the original location of Record 8, they must be updated synchronously; otherwise, the indices will become invalid.

Problem 3:

(1) Since $n = 4$, internal node: up to 3 keys, 4 pointers, leaf node: up to 3 keys.

Insert 5 $\rightarrow$ [5]

Insert 7 $\rightarrow$ [5, 7]

Insert 2 $\rightarrow$ [2, 5, 7] (still fits)

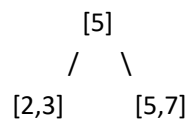
Insert 3 $\rightarrow$ overflow $\rightarrow$ split:

Leaf splits into:

Left: [2, 3]

Right: [5, 7]

Promote 5 to root then we have:



Insert 11 $\rightarrow$ goes to right leaf $\rightarrow$ [5,7,11]

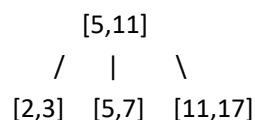
Insert 17 $\rightarrow$ overflow $\rightarrow$ split right leaf:

Right leaf [5,7,11,17] $\rightarrow$ split into:

Left: [5,7]

Right: [11,17]

Promote 11 to root then we have:



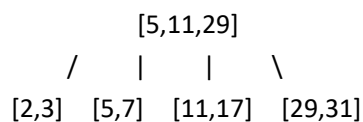
Insert 29 $\rightarrow$ goes to last leaf $\rightarrow$ [11,17,29]

Insert 31 $\rightarrow$ overflow $\rightarrow$ split [11,17,29,31] into:

Left: [11,17]

Right: [29,31]

Promote 29 to root then we have:



Insert 19 $\rightarrow$ goes to leaf [11,17] $\rightarrow$ becomes [11,17,19]

Insert 23 $\rightarrow$ overflow $\rightarrow$ split [11,17,19,23] into:

Left: [11,17]

Right: [19,23]

Promote 19 to root.

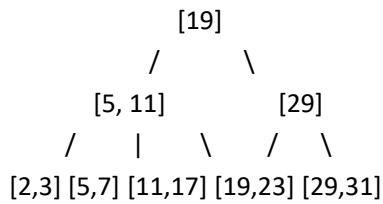
Root overflows: [5,11,19,29] $\rightarrow$ split root:

Left: [5,11]

Right: [19,29]

New root: [19]

Final B⁺ Tree ($n = 4$):



(2) Since $n = 6$, internal node: up to 5 keys, 6 pointers, leaf node: up to 5 keys.

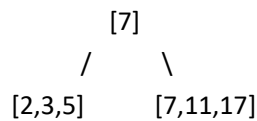
After inserting 5,7,2,3,11 → leaf becomes [2,3,5,7,11] (fits)

Insert 17 → overflow (6 keys) → split [2,3,5,7,11,17] into:

Left: [2,3,5]

Right: [7,11,17]

Promote 7 and we have:



Insert 29 → goes to right leaf → [7,11,17,29]

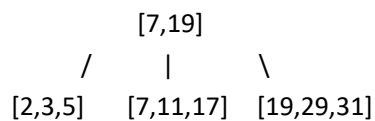
Insert 31 → [7,11,17,29,31] (fits)

Insert 19 → [7,11,17,19,29,31] → overflow → split [7,11,17,19,29,31] into:

Left: [7,11,17]

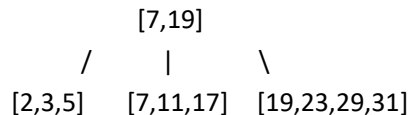
Right: [19,29,31]

Promote 19 and we have:



Insert 23 → goes to right leaf → [19,23,29,31] (fits)

Final B⁺ Tree ($n = 6$):



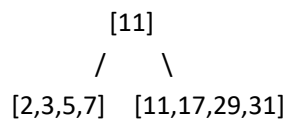
(3) Since $n = 8$, internal node: up to 7 keys, 8 pointers, leaf node: up to 7 keys.

After inserting 5,7,2,3,11,17,29,31 → overflow (8 keys) → split [2,3,5,7,11,17,29,31] into:

Left: [2,3,5,7]

Right: [11,17,29,31]

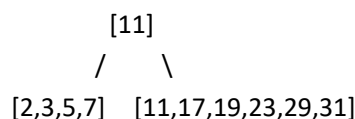
Promote 11 and we have:



Insert 19 → goes to right leaf → [11,17,19,29,31] (fits)

Insert 23 → goes to right leaf → [11,17,19,23,29,31] (fits)

Final B⁺ Tree ($n = 8$):



Problem 4:

(1) ①Start at the root:

Root contains [19].

Since $11 < 19$, follow the left pointer.

②Go to the left internal interval: [5, 11].

$11 = 11 \rightarrow$ follow the pointer to the third child.

③Arrive at the leaf node:

Leaf contains [11, 17].

The key 11 is found in the interval: [11, 17]

(2) ①Start at the root:

$7 < 19 \rightarrow$ go to the left subtree.

②At internal node [5,11]:

$5 \leq 7 < 11 \rightarrow$ follow the middle pointer.

③Arrive at leaf [5,7]:

$7 = 7 \rightarrow 7$ is included.

④Move to the next leaf (right sibling):

Leaf: [11,17]

Both 11 and 17 fall within the range.

⑤Next leaf is [19,23], but $19 > 17 \rightarrow$ stop.

All keys in the range 7 to 17 inclusive: 7, 11, 17.

These are found across two adjacent leaf nodes: [5,7] and [11,17].